

Manual Técnico del Desarrollador: Sistema de Inventario

Organización: Honda Avante

Módulo: Gestión de Refacciones y Movimientos

Desarrollador Principal: Antonio Celis correo: 'antcelis97@gmail.com'

Fecha: 20/07/2026

1. Arquitectura del Sistema

El proyecto opera bajo una arquitectura cliente-servidor completamente desacoplada que se comunica vía API REST.

- **Frontend (Cliente):** Desarrollado en HTML5, Vanilla JavaScript y Tailwind CSS. Sigue una estructura modular que inyecta componentes de forma dinámica para simular una Single Page Application (SPA).
- **Backend (Servidor):** Construido sobre PHP utilizando el framework Laravel.
- **Base de Datos:** PostgreSQL relacional.

Para que el entorno local empate perfectamente con producción y no haya problemas de compatibilidad, se deben respetar las siguientes versiones:

A. Backend

- **PHP:** Versión 8.5 (Requerida según la imagen php:8.5-apache en Docker).
- **Base de Datos:** PostgreSQL 17 o superior.
- **Composer:** Última versión estable.
- **Extensiones PHP vitales:** pdo_pgsql, gd (procesamiento de imágenes), zip.

B. Frontend

El frontend no requiere Node.js instalado obligatoriamente ya que carga mediante CDN, pero depende de las siguientes librerías en sus versiones específicas:

- **Tailwind CSS:** Vía CDN (configurado en el <head>).
- **SweetAlert2:** v11 (Alertas y modales).
- **Browser Image Compression:** v2.0.2 (Gestor de compresión webp).

- **FontAwesome:** v6.4.0 (Iconografía).

2. Configuración del Entorno Local

Para levantar el proyecto en un entorno de desarrollo, sigue estos pasos:

1. Clonar el repositorio del backend y frontend.
2. **Variables de Entorno (Backend):**
 - Copiar el archivo .env.example y renombrarlo a .env.
 - Configurar las credenciales de la base de datos PostgreSQL (DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD).
 - Generar la clave de la aplicación ejecutando: `php artisan key:generate`.
 - Generar la clave secreta para los tokens de seguridad: `php artisan jwt:secret`.
3. **Dependencias:** Ejecutar `composer install` para descargar las librerías de PHP.
4. **Base de Datos:** Ejecutar `php artisan migrate` para crear las tablas, y `php artisan db:seed` (si existen seeders) para poblar datos iniciales.

3. Estructura del Backend (Laravel)

El núcleo del servidor utiliza el patrón de diseño MVC (Modelo-Vista-Controlador), omitiendo las "Vistas" nativas de Laravel para fungir estrictamente como una API REST.

A. Migraciones y Base de Datos

Las migraciones (`database/migrations/`) son el control de versiones de la base de datos.

- **Modificar estructuras:** NUNCA se debe modificar la base de datos directamente en el gestor SQL. Para agregar o quitar columnas, se debe crear una nueva migración con `php artisan make:migration nombre_de_migracion` y definir los cambios allí.

B. Modelos y Relaciones

Ubicados en `app/Models/`. Representan las tablas de la base de datos (ej. `SparePart`, `Category`, `Movement`).

- Aquí se definen las relaciones de Eloquent ORM (ej. `belongsTo`, `hasMany`). Las llaves foráneas deben respetarse estrictamente para mantener la integridad referencial.

C. Controladores y Rutas

- **Controladores** (`app/Http/Controllers/`): Contienen la lógica de negocio. Por ejemplo, `SparePartController` gestiona el CRUD del inventario.
- **Rutas** (`routes/api.php`): Todos los endpoints disponibles para el frontend se registran aquí. Están protegidos por middlewares de autenticación.

D. Seguridad y Autenticación (JWT)

El sistema no utiliza sesiones tradicionales. Se implementó JSON Web Tokens (JWT).

- Al iniciar sesión, el servidor devuelve un Token encriptado.
- Todas las rutas de la API en `routes/api.php` están envueltas en el middleware `auth:api`, lo que significa que el servidor rechazará cualquier petición que no incluya un Token válido en las cabeceras HTTP (`Authorization: Bearer <token>`).

E. Comandos Artisan Esenciales

- `php artisan serve`: Levanta el servidor local.
- `php artisan migrate:fresh`: Destruye toda la base de datos y la vuelve a crear desde cero (Usar solo en desarrollo).
- `php artisan storage:link`: Crea el puente simbólico necesario para que las imágenes almacenadas en el backend sean visibles públicamente en la web.

F. Bot Sincronizador

F. Integración con POS Externa (API & CSV)

El sistema no utiliza tareas programadas (Cron Jobs) internas. En su lugar, actúa como un servidor receptor para integrarse con el Punto de Venta (POS) desarrollado en Node.js. Toda esta lógica está centralizada en el controlador `PosIntegrationController`.

Existen dos vías principales de sincronización:

- Plan A (API a API): El sistema expone rutas protegidas que reciben *payloads* en formato JSON.
 - syncMovementsAPI: Recibe arreglos de movimientos (entradas/salidas) y ajusta el current_stock automáticamente.
 - syncCatalogAPI: Recibe productos nuevos desde el POS. Si la categoría o proveedor no existen, el sistema los crea dinámicamente ("Categoría General" / "Proveedor General") para evitar errores de llaves foráneas.
- Plan B (Respaldo CSV): Si la comunicación con Node.js falla, el sistema cuenta con el método importMovementsCSV, que permite al administrador subir un archivo plano con 5 columnas (SKU, Tipo, Cantidad, Motivo, Fecha) para sincronizar los movimientos de manera manual.

4. Estructura del Frontend (Vanilla JS)

El frontend está diseñado para ser ligero, rápido y no depender de frameworks pesados como React o Angular, manteniendo una alta escalabilidad.

A. Componentes Universales y Carga Dinámica

Para no repetir código HTML en cada pantalla, se utiliza el archivo component-loader.js.

- Este script lee marcadores vacíos en el HTML (ej. <div id="navbar-placeholder"></div>) y utiliza fetch() para inyectar fragmentos de código desde la carpeta components/.
- Esto permite que cambios en la barra de navegación o modales globales se hagan en un solo archivo y se reflejen en todo el sistema.

B. Configuración de Endpoints

Todas las peticiones a la API apuntan a una variable global definida en assets/js/config.js.

- **Variable** window.APP_API_URL: Permite cambiar el destino de todas las peticiones con una sola línea de código (de localhost para desarrollo, a la URL del servidor de producción).

C. Peticiones (Fetch API) y Seguridad

Todas las transacciones con el servidor se realizan mediante la función nativa `fetch()`.

- El token JWT devuelto al iniciar sesión se almacena temporalmente en el `localStorage` del navegador (`localStorage.getItem('honda_token')`).
- Cada petición `fetch()` incluye manualmente este token en la cabecera `'Authorization': 'Bearer ' + token` para superar el bloqueo de seguridad de Laravel.

5. Lógica Core y Reglas de Negocio

Funciones críticas que el desarrollador debe conocer al modificar o expandir el sistema.

A. Procesamiento de Imágenes

El sistema comprime las fotografías de las refacciones directamente en el navegador del cliente antes de enviarlas al servidor.

- **Herramienta:** `browser-image-compression`.
- **Parámetros:** Se transforman al formato `image/webp` con un peso máximo de 0.5MB y resolución máxima de 800px. Si la imagen es menor a estos parámetros, la compresión se omite automáticamente para ahorrar recursos.

B. Importación y Extracción Masiva (CSV)

El módulo de importación masiva permite registrar múltiples refacciones simultáneamente.

- **Estructura estricta:** El archivo `.csv` no debe contener fila de encabezados.
- **Orden de columnas (8):** SKU, Nombre, Stock, Precio Compra, Precio Venta, ID Categoría, ID Proveedor, ID Subcategoría.
- **Llaves foráneas:** Las categorías, proveedores y subcategorías deben enviarse como IDs numéricos. Enviar cadenas de texto o IDs inexistentes provocará un error de "Foreign Key Violation" en PostgreSQL y abortará la operación.

C. Cálculo Financiero Dinámico

El sistema no requiere que el operador defina el precio de venta final de forma manual. Este se calcula en tiempo real mediante la función `calcularPrecioVenta()` en el frontend tomando en cuenta:

1. Precio de compra base de la refacción.
2. Porcentaje de margen de utilidad (Regla de negocio: se emite una alerta si es menor al 25%).
3. Impuestos (IVA).
4. Descuentos u ofertas aplicables.

6. Administración de Base de Datos y Usuarios

A. !!! Vaciar la Base de Datos (Limpieza) !!!

Durante la fase de pruebas o al preparar el sistema para un nuevo ciclo, puede ser necesario limpiar los registros.

- **Destrucción Total:** Ejecutar `php artisan migrate:fresh` eliminará todas las tablas (incluyendo usuarios) y las volverá a crear vacías.
- **Limpieza Selectiva (Conservando Usuarios):** Para borrar solo el inventario y movimientos, pero mantener al personal y sus credenciales intactas, se debe utilizar la consola interactiva de Laravel (Tinker) o un script SQL directo.

Opción con Laravel Tinker:

Ejecutar `php artisan tinker` en la terminal del servidor y pegar lo siguiente:

PHP

```
Schema::disableForeignKeyConstraints();
```

```
DB::table('spare_parts')->truncate();
```

```
DB::table('movements')->truncate();
```

```
DB::table('categories')->truncate();
```

```
DB::table('subcategories')->truncate();
```

```
DB::table('proveedores')->truncate();
```

```
Schema::enableForeignKeyConstraints();
```

Opción con SQL (vía pgAdmin o cliente SQL):

SQL

TRUNCATE TABLE spare_parts, movements, categories, subcategories, proveedores
RESTART IDENTITY CASCADE;

(Nota: RESTART IDENTITY asegura que los IDs vuelvan a comenzar desde el número 1).

B. Gestión de Contraseñas

Si un usuario olvida su contraseña o se requiere un reseteo general por seguridad, se utiliza Laravel Tinker (php artisan tinker):

- **Cambiar contraseña de un usuario específico:**

PHP

```
$user = App\Models\User::where('email', 'operador@honda.com')->first();
```

```
$user->password = Hash::make('NuevaClave123*');
```

```
$user->save();
```

- **Restablecer la contraseña de TODOS los usuarios (Emergencia):**

PHP

```
App\Models\User::query()->update(['password' =>
```

```
Hash::make('HondaAvante2026*')]);
```

7. Herramientas de Apoyo: pgAdmin

Se recomienda encarecidamente que el desarrollador instale **pgAdmin 4** en su equipo local y se conecte a la base de datos de producción utilizando las credenciales .env.

- **Utilidad principal:** Permite visualizar el Diagrama Entidad-Relación (ERD) en tiempo real, lo que facilita entender cómo se conectan las llaves foráneas entre tablas.
- **Soporte:** Es más seguro y rápido para realizar exportaciones masivas a Excel de tablas completas, o para forzar la eliminación de registros problemáticos que bloquean las llaves foráneas.

8. Plan de Mantenimiento y Escalabilidad

Para asegurar que el sistema de inventario sobreviva al paso del tiempo sin degradación de rendimiento, se recomiendan las siguientes acciones:

A. Corto Plazo (Semanal/Mensual)

- **Monitoreo de Servidores:** Revisar los logs nativos del servidor (Apache/Nginx o panel de control) para identificar si alguna petición fetch está fallando por timeout (tiempos de espera excedidos).
- **Almacenamiento y Recursos:** Monitorear el espacio en disco del servidor VPS o dedicado. Aunque las refacciones de texto ocupan poco espacio, es una buena práctica mantener vigilancia sobre el crecimiento constante de las imágenes.

B. Mediano Plazo (Semestral)

- **Backups Manuales:** Configurar tareas cron para respaldos automatizados o ejecutar periódicamente un `pg_dump` vía terminal para tener copias de la base de datos guardadas de manera local u offline en los servidores de la agencia.
- **Depuración de Movimientos:** Si la tabla `movements` crece a cientos de miles de registros, considerar exportar los movimientos de años anteriores a un CSV histórico y limpiar la tabla para mantener la velocidad de las consultas.

C. Largo Plazo (Escalabilidad de Imágenes)

- **Migración de Storage:** Actualmente, las fotografías se almacenan en el disco local del servidor de alojamiento (`/storage/app/public`). Para proyectos de nivel corporativo en crecimiento, se recomienda modificar el archivo `config/filesystems.php` de Laravel para conectar el almacenamiento de imágenes directamente a un *Bucket* persistente en la nube (como AWS S3). De esta forma, el servidor web y el almacenamiento de archivos quedarán 100% desacoplados, optimizando la velocidad y facilitando migraciones futuras de servidor.

9. Hoja de Ruta y Mejoras Futuras (Roadmap v2.0)

El sistema actual opera bajo un modelo de sincronización reactiva (basado estrictamente en endpoints y peticiones del POS). Para escalar el nivel de automatización y robustecer la operativa diaria del almacén, se recomienda

encarecidamente implementar el **Programador de Tareas de Laravel (Task Scheduler)** mediante *Cron Jobs*.

Evolucionar hacia un modelo "proactivo" reducirá la dependencia humana y mitigará riesgos por cortes de red.

A. Casos de Uso Propuestos para Automatización

Se recomienda desarrollar comandos en Laravel (php artisan make:command) para las siguientes tareas operativas:

- **Alertas de Stock Crítico:** Un comando que se ejecute diariamente a las 07:00 AM para escanear las refacciones cuyo current_stock haya caído por debajo del min_stock, enviando un correo electrónico automatizado al gerente con la lista de resurtido del día.
- **Conciliación Nocturna:** Un bot programado a las 03:00 AM que consulte la base de datos central y verifique si existen discrepancias de inventario que no se hayan reportado a través de la API durante el día por fallas de conexión.
- **Reportes de Rendimiento Automáticos:** Generar un reporte en Excel o PDF con el resumen de movimientos (entradas y salidas) y enviarlo por correo a gerencia cada lunes a primera hora.
- **Mantenimiento Interno:** Tarea dominical para limpiar registros obsoletos (como tokens JWT expirados o archivos temporales no procesados), garantizando la velocidad de respuesta de PostgreSQL a largo plazo.

B. Consideraciones Técnicas de Implementación

Para que el desarrollador implemente esta automatización, debe seguir estas directrices:

- **Configuración del Servidor:** La elegancia del Scheduler de Laravel permite que solo se requiera configurar una única entrada en el crontab del servidor Linux de producción. Todas las tareas subsecuentes se programan en app/Console/Kernel.php.

Bash

```
* * * * * cd /ruta-a-tu-proyecto && php artisan schedule:run >> /dev/null 2>&1
```

- **Prevención de Colisiones (Critical):** Si se implementa un bot que sincronice o audite datos masivos, es imperativo encadenar el método ->withoutOverlapping() a la definición de la tarea. Esto evita que, si un proceso

sufre un retraso por el volumen de datos, el siguiente ciclo inicie antes de que el primero termine, evitando la corrupción de registros.

C. Integración con Sistemas Financieros y ERP

El siguiente paso estratégico para la madurez del sistema es convertirlo en una fuente de verdad para el departamento contable, estableciendo un puente de comunicación (vía API, Webhooks o reportes automatizados) con el software de finanzas actual de la agencia.

Se sugiere desarrollar módulos o *endpoints* específicos que procesen y exporten la siguiente información financiera:

- **Registro de Compras:** Sincronización de los costos reales de adquisición de refacciones, facilitando la conciliación con las facturas de proveedores.
- **Análisis de Utilidad:** Reportes dinámicos del margen de ganancia obtenido por periodo, cruzando el precio de compra histórico contra el precio de venta final de las piezas.
- **Clasificación de Salidas (Merma y Taller):** Diferenciar contablemente las salidas por venta al público de las salidas por *merma* (piezas defectuosas/dañadas) o *uso interno en taller*. Esto permitirá al departamento de finanzas deducir costos o asignar presupuestos internos de manera precisa.
- **Devoluciones sobre Compra:** Automatizar el ajuste de capital cuando se retorna mercancía al proveedor, facilitando el cuadre de notas de crédito.
- **Valorización de Capital de Inventario:** Un servicio que calcule en tiempo real el valor total monetario inmovilizado en el almacén (`current_stock` multiplicado por el `purchase_price` promedio), una métrica vital para los cierres de mes contables.

Nota para el desarrollador: La implementación técnica de esta fase puede apoyarse en las tareas programadas (*Cron Jobs*) mencionadas anteriormente para exportar "cortes de caja" nocturnos, o mediante la emisión de eventos de *Laravel* (*Events/Listeners*) que envíen datos al sistema financiero en tiempo real cada que ocurra una transacción crítica en el catálogo.